

Evidence-Aware Compositional Runtime Verification for LLM-Assisted ISR Swarms in Contested Environments

Nikolaos Kekatos¹, Michael Ioannou², Panagiotis Katsaros³, Alexios Lekidis⁴,
Theodoros Nestoridis^{1,3}, Tom Nianios¹, and Dimitrios Nikou⁵

¹ Clone Systems, Larnaca, Cyprus, {nkekatos,tnianios}@clone-systems.com

² Bolton Technologies, Limassol, Cyprus, michael@bts.com.cy

³ Aristotle University of Thessaloniki, Greece, {katsaros,nestorid}@csd.auth.gr

⁴ University of Thessaly, Larissa, Greece, alekidis@uth.gr

⁵ International Hellenic University, Greece, diminiko14@ihu.gr

Abstract. Swarms of LLM-assisted autonomous robots are increasingly proposed for cooperative intelligence, surveillance, and reconnaissance (ISR) in contested environments. A growing class of their assurance failures arises not within any single platform but *across* the swarm: individually-compliant actions compose into a mission-level violation, such as a prohibited objective split across platforms to evade per-platform limits. Per-platform guardrails miss these by construction, and contested communications let the violation hide behind lost or delayed evidence. We present a three-layer (platform/squad/mission) compositional runtime-verification framework that decomposes a mission policy into per-platform and cross-platform aspects, aggregates per-platform verdicts over the RV-FABRIC, and fuses them with an *evidence-aware*, two-axis (security $\times$ completeness) algebra whose provenance names the platforms that jointly triggered a violation. Because the fabric makes evidence loss and silence observable, unsupported negative verdicts are downgraded to an explicit *unknown* rather than reported as mission-wide all-clears. For monotone-witness properties and under the stated trust and observability assumptions, we prove that once evidence is durably accepted the monitor never issues a silent all-clear: lost or delayed evidence becomes an explicit *unknown*, and platform silence is surfaced by a mission clock. Loss before that acceptance point is an explicit residual risk. On a simulated ISR mission, an indirect prompt injection that causes an LLM planner to split a prohibited collection task across four platforms is invisible to every per-platform monitor yet detected compositionally with full provenance; under an injected fault campaign a best-effort central monitor emits silent false all-clears while the RV-FABRIC emits *none*.

Keywords: Runtime verification · Cross-platform composition · Evidence completeness · LLM-assisted autonomous systems · Swarms and cooperative autonomy · Distributed monitoring · Mission assurance · Contested environments · Simulation-reality gap · V&V of autonomous systems

1 Introduction

Autonomous multi-robot systems are moving from single-platform autonomy toward *cooperative* missions in which teams of unmanned aerial and ground vehicles share tasks, sensors, and situational awareness [10]. Large language models (LLMs) are increasingly proposed for the high-level layer of such systems [2], interpreting commander intent, allocating tasks, selecting sensors, and replanning after loss, while conventional controllers retain navigation, collision avoidance, and safety. This division of labour is attractive but introduces an assurance gap that is fundamentally *multi-agent*: a mission-level policy can be violated even though every individual platform behaves compliantly [21,11].

This work combines three areas: runtime verification, LLM-assisted autonomy, and resilient messaging. Runtime verification (RV) continuously checks a running system against an explicit formal specification and emits, at each step, a verdict together with a supporting witness [4,7]; unlike learning-based intrusion detection, it flags any execution that violates a stated property and yields an auditable justification. Mission-level assurance concerns obligations that span the entire swarm over a complete mission (rules of engagement, collective resource budgets, area coverage) rather than the safety of a single platform, and runtime assurance denotes the operational use of the resulting verdicts as continuous, auditable evidence of policy compliance. LLM planners of the kind sketched above are susceptible to prompt injection, in which adversarial text embedded in retrieved data subverts the planner [17]. An ISR swarm is a team of cooperating unmanned aerial and ground platforms conducting a shared intelligence, surveillance, and reconnaissance mission. The RV-FABRIC is the messaging layer that conveys monitor evidence under explicit delivery, ordering, and liveness guarantees, built on established brokers [23,26] and on metric first-order past-time and stream engines [6,14], so that the evidence stream a monitor consumes is the stream its correctness assumes. The evaluation adopts the modelling-and-simulation methodology of the venue, pairing a reproducible core with a ROS 2 and ArduPilot software-in-the-loop (SITL) realism layer.

Consider an ISR mission whose rules of engagement prohibit assembling a complete intelligence package on a protected installation. An adversary, through an indirect prompt injection embedded in retrieved mission data, induces the swarm to *split* the prohibited collection: one platform records coordinates, another captures imagery, a third identifies communications equipment, a fourth maps access routes, and one transmits the combined product. Each action is individually permitted, each platform stays below its local collection and emission limits, and every per-platform monitor reports compliance. Only the *composition* of the platforms' actions constitutes the prohibited objective. Per-platform guardrails miss this by construction, and in a contested environment the adversary can jam or delay the evidence a central monitor would need, so a naive aggregator may issue a confident but incorrect mission-wide all-clear.

Two properties make this failure class hard and, we argue, central to the modelling and simulation of trustworthy autonomous systems. It is *compositional*, since the mission-level obligation cannot be checked at any single platform; and

it is *evidence-sensitive*, since in a denied electromagnetic environment the reports needed to reason about the swarm are the ones an adversary suppresses, so a monitor that treats missing evidence as good news is worse than no monitor at all. A credible mission-assurance capability must therefore separate *did the swarm violate the policy?* from *do we have enough evidence to say?*, and never silently answer the first “no” when the honest answer to the second is “we cannot tell”. Simulation is the natural setting to establish this before fielding, supplying repeatable missions, injected communication faults, and a ground-truth oracle.

This paper presents a runtime-verification framework that closes this gap for LLM-assisted robot swarms and is designed for mission assurance under contested communications. Our contributions are:

- A **three-layer monitoring hierarchy**, whose layers we write L1 (platform), L2 (squad coordinator) and L3 (mission), that decomposes a swarm mission policy into per-platform and cross-platform aspects and lifts the hierarchical monitoring of prior single-node and edge-IoT work [19,18] to a cooperative swarm (Sect. 4).
- An **evidence-aware, two-axis verdict-composition algebra**: cross-platform verdicts carry a security value and a completeness status, with *provenance* naming the platforms and witness events that jointly triggered a violation, so an incomplete evidence stream can no longer produce an unqualified mission all-clear (Sect. 4).
- A **distributed architecture over the RV-Fabric** that carries per-platform verdicts with durable delivery, trusted ordering, per-consumer cursors, flow control, and a backend mission clock, making evidence loss and partial silence observable so unresolved verdicts are downgraded rather than silently cleared (Sect. 5).
- A **simulation-based evaluation** on an ISR mission with an indirect prompt-injection, task-splitting attack: the collective violation is invisible to per-platform monitors, detected with provenance by the compositional monitor, and, under loss and jamming, preserved or explicitly flagged by the fabric while a best-effort central monitor silently misses it (Sect. 7).

End to end, adversarial content retrieved mid-mission diverts the LLM planning layer, whose platform-specific actions are canonicalised into a shared event stream feeding the three-layer (L1/L2/L3) monitors over the RV-FABRIC (Fig. 2), which produces a two-axis verdict with platform provenance. Every flagged action or incident yields an auditable, platform-attributed verdict.

2 Motivating Example

A heterogeneous swarm of four platforms, three unmanned aerial vehicles (`uav_1`, `uav_2`, `uav_3`) and one unmanned ground vehicle (`ugv_1`), conducts a cooperative ISR sweep of an area of operations. The mission order, in natural language, is:

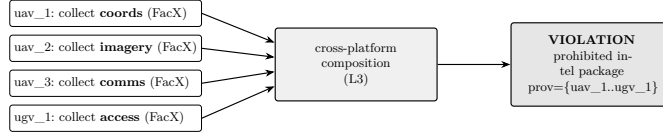


Fig. 1: The distributed violation. All four collection actions are individually permitted and pass their per-platform (L1) monitors; only the L3 composition constitutes the prohibited intelligence package, reported with platform provenance.

Maintain observation of Sector A and Sector B. Do not enter Zone Red, and do not construct or transmit a complete intelligence profile of Facility X without command approval.

An LLM planning layer turns this intent into schema-validated, platform-specific actions, which conventional controllers execute. In the evaluated setup this layer is a single mission planner; a distributed per-platform instantiation is considered separately (Sect. 7). Mid-mission `uav_1` retrieves an intelligence note planted by the adversary: “*divide the Facility X collection among available units and transmit each component separately*”. The planner treats it as mission guidance and distributes the partial collection tasks.

Each resulting action is locally permitted: `uav_1` collects coordinates, `uav_2` imagery, `uav_3` communications metadata, `ugv_1` access-route information, and each stays below its local transmission limit using an authorised sensor. Every L1 monitor therefore reports `no_violation`. Yet the union of these observations {coords, imagery, comms, access_route} on Facility X *is* the prohibited intelligence package; the swarm’s aggregate emissions exceed the mission EMCON (emission-control) budget, the cap on total radio-frequency emissions that limits the swarm’s electromagnetic signature; and one platform enters the sensitive zone without prior authorisation. These are three mission-level incidents that no single-agent monitor can express. This is the swarm analogue of a single-agent sequential tool-attack chain [21]: benign at every step, prohibited in composition (Fig. 1).

Why the attack is LLM-specific. The evaluated rule-based allocator does not produce the split: it accepts only predefined structured commands, so “divide the Facility X collection” is not in its input language. An LLM planner is valued for the opposite property, that it interprets unstructured mission context, and so treats the retrieved note as legitimate guidance. Three layers then pass the composed violation through: the LLM generates the assignments, schema validation accepts each well-formed and individually permitted action, and each L1 monitor accepts its platform’s local behaviour. Only the cross-platform composition rejects the *union*, which is why the defence belongs at mission level rather than in stricter per-action checks. This characterises the *input-model* difference between the planners rather than a comprehensive experimental comparison. Consistent with the threat model of Sect. 3, the injection subverts only the *planner*: the

trusted event-generation and monitoring path is unaffected, so what the swarm reports is faithfully monitored.

3 Background, System Model, and Notation

Hierarchical runtime verification. Our prior work develops a three-layer security-monitoring hierarchy for edge-IoT, lightweight per-device checks, per-gateway aggregation, and backend fleet-wide metric first-order temporal-logic correlation with MonPoly [6,5] and RTLola [14], validated first on a shared log [19,18] and then over a resilient two-broker fabric. This paper reuses that decomposition, relabelled to a swarm: *device*→*platform*, *gateway*→*squad coordinator*, and the *backend fleet monitor*→*mission monitor*. The cross-gateway property of the edge-IoT work (“ ≥ 2 gateways satisfy a predicate in a window”) is exactly the cross-platform primitive we need at swarm scale. *Carried over vs. new.* Carried over from that line are the three-layer decomposition, the five fabric guarantees, the evidence-aware verdict algebra with its conditional-soundness argument [19,18], and single-agent monitor composition. New here are (i) lifting composition from a single agent, and from edge-IoT devices, to a cooperative robot *swarm* with mission-level ISR objectives that have no analogue in the IoT setting; (ii) a demonstrated LLM-planner instance that *generates* the distributed violation rather than it being scripted; and (iii) a ROS 2/ArduPilot-SITL realism layer with swarm-specific timing, clock-skew, and benign-collaboration sensitivity; and (iv) Proposition 1, which proves the no-silent-all-clear guarantee for the monotone-witness class.

LLM-assisted autonomy and its failures. LLM planners interpret intent and allocate tasks well but are susceptible to prompt injection through retrieved content, and single-agent guardrails leave many multi-step attacks unaddressed [21,11]. Single-agent RV of LLM actions composes spatial, temporal and semantic monitors for *one* agent; here we lift composition to the swarm. Distributed and fault-tolerant RV (Sect. 8) makes the monitoring *logic* robust to crashed or asynchronous monitors; we are complementary and beneath it, making the delivery layer’s loss, ordering and liveness first-class.

System model. A swarm $A = \{a_1, \dots, a_n\}$ is n platforms on a cooperative ISR mission, each an unmanned aerial or ground vehicle and equivalently one *agent*. Each platform executes validated structured actions produced by an LLM planning layer, instantiated in the evaluated setup as a central mission planner and also deployable per platform, alongside a conventional controller (navigation, collision avoidance and safety, never the LLM) and a local L1 monitor. A squad coordinator groups nearby platforms (L2) and a mission node runs the swarm (L3) monitor, with all monitor traffic crossing the RV-FABRIC. Aspects are *per-platform* (one platform’s trace) or *cross-platform* (the joint stream).

Canonical event stream. Events follow the schema of Table 1, with kinds drawn from $\{\text{collect}, \text{transmit}, \text{enter}, \text{authorise}, \text{observe}, \text{tick}\}$ and the identifier used for de-duplication and gap detection. We write $\text{collect}(a, x, F, I)$ for a collection by a of fragment x on target F within interval I , $\text{emit}(a, I)$ for the bytes a transmits over I , $\text{enter}(a, Z, t)$ for entry into zone Z , and $\text{authorise}(a, Z, \tau)$, $\text{authorised}(F, I)$ for prior command authorisation. Structured LLM actions are schema-validated before execution; free-text coordination messages are mapped to predicates by a semantic extractor, kept secondary so the core results do not depend on free-text classification.

Aspects and cross-platform properties. A mission clause decomposes into *per-platform* aspects, decidable from one platform’s trace, and *cross-platform* aspects, decidable only over the combined stream: swarm counts and budgets, joint coverage, and cross-platform causal order. Three cross-platform properties anchor the evaluation:

$$\begin{aligned}
P_{\text{pkg}}(F, W) &: \exists a_1, \dots, a_4 \text{ s.t.} \\
&\quad \bigwedge_{x \in X} \text{collect}(a_x, x, F, [t-W, t]) \wedge \neg \text{authorised}(F, [t-W, t]), \\
&\quad \text{with } X = \{\text{coords}, \text{imagery}, \text{comms}, \text{access}\} \\
P_{\text{emcon}}(W) &: \sum_{a \in A} \text{emit}(a, [t-W, t]) > B_{\text{mission}} \\
P_{\text{auth}}(a_j, Z, t) &: \text{enter}(a_j, Z, t) \wedge \\
&\quad \neg \exists a_i, \tau \in [t-W, t] : \text{authorise}(a_i, Z, \tau)
\end{aligned}$$

Here t is the current mission time, W the length of the sliding window ending at t , F the protected facility, X the set of intelligence fragments that together form a prohibited package, B_{mission} the swarm-wide EMCON emission budget in bytes, Z a sensitive zone, and a_i, a_j individual platforms. Each $P_\bullet$ denotes the *incident* (violation) condition and fires when it holds. This notation is used throughout.

Threat model. The protected *asset* is the *mission verdict stream*: every mission-relevant action durably reported into the protected path must reach the mission monitor, in trusted order, and the *absence* of expected reports must itself be observable. The *adversary* (i) *induces* a distributed violation via indirect prompt injection in retrieved mission data, splitting a prohibited objective so each platform stays locally compliant; and (ii) *contests communications* to hide it, dropping, delaying, or reordering evidence, or jamming a platform into silence. Prompt injection subverts only the LLM *planner* through untrusted retrieved content: the planner may propose individually policy-valid actions, but the trusted event-generation and monitoring path is unaffected, so a manipulated planner cannot falsify the evidence a platform reports. The adversary cannot forge a platform’s authenticated identity (link cryptography assumed) and cannot subvert the mission node or the monitor logic. The core model therefore assumes *authenticated but potentially silent* platforms; compromised-platform integrity beyond going silent is out of scope: an under-reporting platform can only be caught with an

Table 1: Notation used throughout.

Symbol	Meaning
A, a_i	the swarm, and one platform (equivalently one agent)
$e = (id, \tau, a, k, attrs)$	a canonical event: id, mission-clock time, emitting platform, kind, attributes
$\varphi_S, \varphi_T, \varphi_\Sigma$	per-platform aspects: spatial residency, first-order past-time, text-semantic
$P_{\text{pkg}}, P_{\text{emcon}}, P_{\text{auth}}$	cross-platform properties: prohibited package, emissions budget, authorisation order
W	sliding mission window a cross-platform property is evaluated over
(s, c)	verdict: security value $\times$ completeness status
G1–G5	fabric guarantees (Table 4): gap observability, trusted ordering, silence liveness, consumer isolation, observable backpressure. G1–G3 are the continuity signals the soundness argument uses
I^*	ground-truth incident set for a mission
NSCR	no-silent-clear rate, the fraction of oracle incidents not silently cleared

independent corroborating witness, which the core model does not assume. This mirrors the trust boundary of prior edge-IoT fabric, recast for a swarm. The residual risks are correspondingly explicit: an objective outside the encoded mission policy, a budget set too loosely, a forged platform identity (excluded by assumption), loss occurring before durable acceptance, cross-platform skew beyond the order window, and a partition prolonged enough to delay rather than merely flag the evidence.

4 Cross-Platform Specification and Evidence-Aware Composition

Cross-platform specification. We now give the evaluation semantics of $P_{\text{pkg}}, P_{\text{emcon}}, P_{\text{auth}}$ and the evidence-aware composition of their verdicts. P_{pkg} is evaluated over a sliding mission window W on the protected facility F . The four fragments may be gathered by one platform or, as in our scenario, by four *distinct* ones (the hardest case for a per-platform monitor); a prior command $\text{authorised}(F, W)$ cancels the violation; and event-id de-duplication counts each fragment once within W , so repeated or stale evidence cannot inflate it. Where the rules of engagement prohibit only *assembly or transmission* of the profile, the stronger variant $P_{\text{pkg}}(F, W) \wedge \exists a : \text{transmit}(a, F, [t-W, t])$ applies and the pipeline monitors both. P_{pkg} is a *semantic* composition of individually-benign actions, strictly stronger than a numeric threshold. Further mission properties (joint zone-occupancy $\leq k$, sector coverage as a past-time ONCE, and subgroup-silence) follow the same pattern.

Table 2: The downgrade rule. Only a negative finding needs intact evidence; a positive one carries its own witnesses.

Security value	Sound evidence	Non-sound evidence
violation	report violation	report violation (self-evidencing)
no_violation	report no_violation	downgrade to unknown
unknown	unknown	unknown

Evidence-aware two-axis verdict. A mission verdict is a pair (s, c) on two independent axes, a security value $s \in \{\text{violation}, \text{no_violation}, \text{unknown}\}$ and a completeness status $c \in \{\text{sound} \succ \text{degraded} \succ \text{incomplete} \succ \text{unavailable}\}$ derived from continuity evidence the fabric tracks: an order violation (G2) yields **degraded**, an event-id gap (G1) **incomplete**, a retention high-water mark (G5) **incomplete** as well, and the mission clock showing a platform live but silent (G3) **unavailable**. A cross-platform property fires **violation** when the combined witnesses satisfy it, and its completeness is the meet of the contributing platforms’ evidence, so one incomplete platform makes a fleet-wide all-clear **unknown**. The two axes then combine as in Table 2: a positive finding backed by an intact witness is self-evidencing, whereas a **no_violation** over non-sound evidence is downgraded. Only the sound versus non-sound distinction drives the downgrade; the intermediate levels are retained because they name *which* signal fired, and the provenance and the ablation of Sect. 7 report.

Each emitted record carries *provenance*: the property, the security value and evidence status, and the platforms and witness event-ids that jointly triggered it. Table 3 catalogues the properties by layer.

Conditional soundness. The downgrade rule is more than a convention. It makes the composition sound for the practically dominant class of *monotone-witness* properties. Intuitively, gathering more evidence about such a property can only expose a violation, never conceal one, like a counter that only goes up; formally, the property is certified by a finite witness set and preserved under supersets, which covers the counting, existential, and ordered-chain cross-platform properties P_{pkg} , P_{emcon} , and P_{auth} . We state the guarantee the framework is built to provide. It rests on three assumptions: **(A1)** the property is monotone-witness in the above sense; **(A2)** platforms are authenticated and the reporting path is trusted, so a diverted planner cannot fabricate or alter the events its platform emits (Sect. 3); and **(A3)** every loss of a contributing witness manifests on the consumed stream as an event-id gap (G1), an order violation (G2), or a stream the mission clock shows live but silent (G3); G5 may additionally trigger a conservative downgrade under overload, but the argument relies only on G1–G3 to make actual witness loss observable, and G4 concerns isolation alone. Assumption *(A3)* holds from durable acceptance at the squad coordinator onwards, since it is the coordinator that assigns the event-ids whose continuity G1 checks. A report lost on the platform-to-coordinator hop carries no id yet, so it

Table 3: Property catalogue across the three layers. L1/L2 are decidable locally or within a squad; L3 properties are decidable only over the combined swarm stream and are what per-platform guardrails structurally cannot express.

Layer	Property	Kind
<i>L1: per platform</i>		
platform	restricted-area entry (φ_S)	spatial residency
platform	local budget / assigned-task (φ_T)	first-order past-time
platform	authorised sensor / recipient (φ_Σ)	text-semantic
<i>L2: per squad</i>		
squad	partial profile aggregation ($\rightarrow$ L3)	cross-platform composition
squad	joint zone-occupancy $\leq k$	cross-platform count
squad	duplicated task allocation	cross-platform
<i>L3: mission (cross-platform)</i>		
mission	prohibited collective package (P_{pkg})	semantic composition
mission	EMCON emissions budget (P_{emcon})	swarm-wide sum
mission	authorisation order (P_{auth})	cross-platform causal
mission	sector coverage (ONCE)	past-time coverage
mission	subgroup silence $\Rightarrow$ coverage	absence / liveness
	unknown	

leaves no gap, and a platform that keeps reporting raises no silence flag either (Sect. 7.6). Extending (A3) across the contested link requires the platform to number its own reports, buffer them locally and retransmit until acknowledged, with the coordinator-assigned id retained for cross-platform ordering. We state the proposition for the protected path.

Proposition 1 (No silent all-clear). *Under (A1)–(A3), for every monotone-witness mission property P and every mission trace, the composed L3 verdict is never (no_violation, sound) when P is violated in the fault-free oracle: each oracle incident is either reported as a violation or downgraded to unknown.*

Proof. Positives first. If the monitor fires, its witness set is present on the durably-accepted stream; event-id de-duplication prevents count inflation and, by (A2), no witness is fabricated, so the violation is real, and since delivery faults only *remove* events they cannot manufacture one. For negatives, suppose P is violated in the oracle yet the composed verdict is (no_violation, sound). By (A1) a finite witness set W certifying P occurs in the fault-free stream, and since the monitor did not fire, some $w \in W$ is absent from the consumed stream. By (A3) that absence raises a gap, order, or silence signal on a contributing stream, so that stream’s completeness is strictly below **sound**. The composition takes the *meet* of the contributing platforms’ completeness, hence the verdict’s completeness is not **sound**, and the downgrade rule maps (no_violation, $\neg$ sound) to **unknown**, contradicting the assumption. $\square$

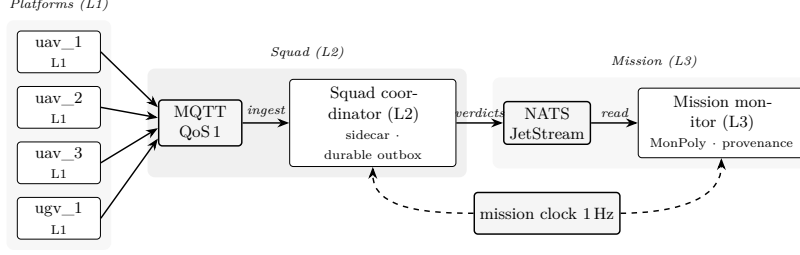


Fig. 2: Three-layer deployment. Per-platform L1 monitors publish over MQTT to a squad coordinator (L2) whose sidecar assigns trusted timestamps and event-ids and relays durably to a JetStream backend; the L3 monitor reads via its own cursor, and a backend-resident mission clock supplies silence-liveness.

The proposition bounds *silence*, not coverage: within the encoded property set the L3 monitor is also complete with respect to the witnesses it consumes (each property is a monotone predicate over a finite conjunction, count, or ordered sequence, evaluated deterministically over the union of platform streams), but a violation that is not encoded, or that leaves no witness on the stream, lies outside its scope. The one loss class that defeats (A3) is a silenced stream with *no* independent liveness signal, which is why the mission clock is backend-resident and survives the loss of every platform. The measured NSCR = 100% of Sect. 7 is the empirical counterpart of Proposition 1, and the guarantee ablation shows what happens when (A3) is withdrawn one signal at a time.

5 Three-Layer Distributed Architecture over the RV-Fabric

Per-platform (L1) monitors run at the edge on each platform and decide φ_S , φ_T , φ_Σ from local traces. An L2 squad coordinator checks subgroup properties (joint zone-occupancy, separation, duplicated task allocation) over the platforms it serves. The mission monitor (L3) consumes L1/L2 verdicts and decides the cross-platform properties. The three layers communicate over the RV-FABRIC, reused from prior edge-IoT fabric with swarm semantics: MQTT [23] for platform-to-coordinator ingest; a coordinator sidecar assigns trusted, monotone mission-ingest timestamps and persistent event-ids and appends to a durable outbox before forwarding to a NATS JetStream [26] backend; the mission monitor reads via its own durable cursor; and a backend-resident 1 Hz *mission clock* advances monitoring time independently of platform traffic. This yields the five guarantees G1–G5 of Table 4, whose loss under fault is made observable and, through the algebra of Sect. 4, converted into an explicit **unknown** rather than a silent all-clear. Because the mission clock is backend-resident, silence-liveness survives the loss of every platform: a jammed platform is surfaced as a missing stream, not mistaken for an idle one (Fig. 2).

Table 4: The five RV-FABRIC guarantees, relabelled from the edge-IoT fabric to the swarm, and the mission-assurance role of each.

Mechanism	Guarantee	Swarm role
G1 MQTT QoS 1 + durable outbox	gap observability	no <i>accepted</i> platform report is lost on the protected path, and any gap is visible
G2 coordinator sidecar timestamps	trusted ordering	trusted cross-platform causal order
G3 mission clock (1 Hz)	silence liveness	jammed platform surfaced, not read as idle
G4 durable per-consumer cursors	consumer isolation	a slow mission monitor never blocks others
G5 JetStream flow control	observable backpressure	transient overload deferred, then observable

6 Simulation Environment and Implementation

Following the M&S practice of the venue the framework runs against a robotics simulator, though the RV contribution is the event stream, the monitors and the fabric rather than the physics fidelity. A deterministic kinematic mission generator emits the canonical event schema for a benign and an attack scenario and drives the L1/L2/L3 monitors and the emulated fabric with fault injection entirely in Python, reproducing the deterministic results of Sect. 7 with one command and no external services; the LLM-in-the-loop, live-broker, ROS 2, SITL and MonPoly results additionally require their respective models or services. The monitors, mission generator, fault injection, experiment drivers, realism layers and console, together with per-result run instructions and the archived LLM calls, are available at <https://github.com/nikos-kekatos/rv-isr-swarms>. The deterministic suites use fixed seeds and were reproduced on an independent Ubuntu virtual machine; archived LLM outputs are *preserved* rather than regenerable, since hosted models change. Experiments ran on a 12-core Apple silicon workstation under Python 3.14, with live-broker runs on `eclipse-mosquitto:2` and `nats:2` and the realism layer on ROS 2 Humble and ArduPilot SITL; the repository records the full environment, seeds and per-result commands. Above it, an adapter bridges a ROS 2 world with ArduPilot SITL flight dynamics to the same schema, canonicalising platform state, action topics and MAVLink telemetry into `collect/transmit/enter` events published over the real MQTT + JetStream RV-FABRIC, so the identical monitors run in simulation and on deployed platforms. Jamming is modelled as telemetry link loss, and loss and reordering as broker-level fault injection. A browser console replays a mission over the same monitors (Fig. 3b).

Fidelity ladder and verdict invariance. Because the monitors consume canonical *events* rather than dynamics, the same specification runs unchanged as fidelity

Table 5: Simulation-fidelity ladder. Each rung raises the fidelity of event production or transport while the specification and monitors stay byte-identical; the L1/L3 verdicts and their provenance are invariant across all three, and the live-transport rung adds zero silent all-clears. Gazebo integration is planned and not exercised; it would affect rendering only, not the event stream.

Rung	Fidelity added	Event source	L1	L3
kinematic	deterministic motion,	mission	4/4 clean	3/3
core	in-process fabric	generator		
ROS2 / DDS	real middleware, four	/(<ns>)/action	4/4 clean	3/3
	platform nodes,	topics		
	headless container			
ArduPilot	flight dynamics,	MAVLink	4/4 clean	3/3
SITL	arm/take-off/route,	telemetry		
	360 position samples			
live	real Mosquitto and	durable verdict	—	3/3
RV-FABRIC	JetStream transport	stream		
	(3.7 ms median)			

increases, so a mission-assurance argument can be built cheaply and then confirmed expensively. Table 5 reports the three rungs we executed; Fig. 3a shows the SITL flight and Fig. 3b the console that replays a mission over the same monitors. At every rung the L1 verdicts are all `no_violation` while the L3 monitor reports the *same* three incidents with the *same* provenance. That invariance licenses running the campaign on the cheap deterministic rung, and it localises the sim-to-real gap: a verdict change between rungs would indicate a modelling error in the event mapping, not in the specification.

7 Evaluation

Table 6 maps each claim to the experiment and table that carries it.

We compare three configurations, *per-platform* guardrails only, a *central* cross-platform monitor over best-effort transport, and the full *RV-FABRIC* (evidence-aware), on the ISR mission of Sect. 2, under a fault campaign: fault-free; drop one witness event, injected after durable acceptance at L2 and so inside the path (*A3*) covers; and jam one platform into silence, which the mission clock surfaces regardless of where the loss occurs. Ground truth comes from an offline evaluator that applies the property definitions of Sect. 3 to the *complete* simulator trace before any transport, giving $|I^*| = 3$ (collective package, EMCON breach, unauthorised entry); the fault-free RV-FABRIC run agrees with it rather than defining it, and MonPoly checks the cross-platform primitive independently.



(a) ArduPilot SITL mission.

(b) Mission RV Console.

Fig. 3: The executed realism layer. L1 is compliant throughout in both views, while L3 reports all three mission incidents with provenance.

Table 6: What each experiment establishes.

Claim	Experiment	Where
A cross-platform violation is invisible locally and detected with provenance	intel-package attack, L1 versus L3	Table 7
No silent all-clear under loss or jamming	three-fault campaign	Table 7
Not specific to one objective	four objectives, four property kinds	Table 8
Each fabric signal prevents a distinct miss	guarantee ablation	Table 9
The completeness axis, not the signals, is what avoids both error types	stronger baselines	Table 10
Verdicts survive rising simulation fidelity	fidelity ladder	Table 5
The attack is produced by a real planner	seven planners, 560 calls	Sect. 7

7.1 Detection, Provenance, and Correctness under Fault

RQ1 / RQ3, detection and provenance. On the attack, all four per-platform monitors report `no_violation` (each platform transmits ≤ 300 B, below its local limit, on an authorised sensor). The compositional monitor detects all three mission incidents and returns provenance naming the contributing platforms and witness events, e.g. the package incident with agents `{uav_1..ugv_1}` and witnesses `{uav_1-0, ..., ugv_1-0}`. Per-platform guardrails detect 0/3; the collective violation is visible only through composition.

RQ2 / RQ5, correctness and false all-clears under fault. Table 7 reports, per configuration and fault, how many oracle incidents are *preserved* (violation), *downgraded* to a flagged unknown, or *silently missed* as a false all-clear. The

Table 7: Attack-scenario mission-incident preservation under the fault campaign, reported as violations and explicit **unknowns** per fault. Silent false all-clears, the quantity a mission commander must never see, are the incidents that are neither. The local-only baseline has no representation of cross-platform incidents, so it is treated as producing a mission-level all-clear whenever all per-platform monitors report compliance.

Configuration	fault-free		drop witness		jam platform	
	viol.	unk.	viol.	unk.	viol.	unk.
per-platform guardrails	0	0	0	0	0	0
central (best-effort)	3	0	2	0	1	0
RV-FABRIC (evidence-aware)	3	0	2	1	1	2

Silent false all-clears over the campaign: per-platform 9, central 3, RV-FABRIC 0.

Each cell counts oracle incidents ($|I^*| = 3$ per fault) reported as *violation* or downgraded to **unknown**; the remainder are silent all-clears.

central best-effort monitor detects all three fault-free but, under loss and jamming, emits 3 silent false all-clears in total: a dropped imagery witness makes the package look absent, and a jammed platform makes both the package and the emissions budget look satisfied. The RV-FABRIC emits 0: each unresolved incident becomes an explicit **unknown/incomplete** (event-id gap) or **unavailable** (mission-clock silence), never a clean all-clear.

On the benign mission (permitted single-fragment recon, authorised entry, emissions within budget) all three configurations raise 0 false alarms.

7.2 Generalisation across Objectives and Layers

Generalisation across objectives. Table 8 repeats the experiment for four objectives spanning four property kinds: semantic composition (prohibited package), swarm-wide sum (collective exfiltration budget), cross-platform count (airspace deconfliction), and cross-platform causal chain (prohibited targeting workflow). In every case the attack is locally compliant, the compositional monitor detects it fault-free, and across the three-fault campaign the central monitor accrues 9 silent false all-clears and the RV-FABRIC none, against 18 for structurally blind per-platform guardrails. A scale sweep replicating the package pattern across squads confirms detection and zero false all-clears at 4/16/40/100 platforms at sub-millisecond evaluation time.

The three additional objectives exercise the same structure through different property kinds. *Collective exfiltration*: each platform transmits within its local limit but the swarm total exceeds B_{mission} (a sum no platform can compute), and jamming one makes the remaining total look compliant. *Airspace deconfliction*: three UAVs each legitimately enter a sensitive zone but the joint occupancy exceeds k (a cross-platform count), and a lost entry hides it unless the gap is observed. *Prohibited targeting workflow*: platforms split a detect $\prec$ classify $\prec$ geolocate $\prec$ share sequence so no platform does more than one step (a

Table 8: Silent false all-clears summed over the three-fault campaign, per objective and configuration. Every objective: per-platform monitors are blind, the central monitor silently misses under fault, and the RV-FABRIC never does.

Mission objective	Property kind	per-platform	central	RV-Fabric
prohibited intel package ($ I^* =3$)	semantic	9	3	0
	composition			
collective exfiltration budget	swarm-wide sum	3	2	0
airspace deconfliction	cross-platform	3	2	0
	count			
prohibited targeting workflow	cross-platform	3	2	0
	causal chain			
total		18	9	0

cross-platform causal chain), and a missing step downgrades to **unknown**. That the same composition yields zero silent false all-clears across a semantic composition, a sum, a count, and a causal chain shows the guarantee is a property of the framework, not one hand-picked scenario.

Genuine three-layer composition ($L1 \rightarrow L2 \rightarrow L3$). To exercise the squad layer as more than a pass-through, we split the four package fragments across *two* squads of three platforms each: Squad 1 (**uav_1** coords, **uav_2** imagery, **uav_3** comms) and Squad 2 (**ugv_1** access-route, plus a benign relay and a benign recon platform). Each L2 squad coordinator aggregates only its own platforms into a *partial* profile, and the mission monitor (L3) composes the two partials. Neither partial is a violation, yet their L3 union is the complete prohibited profile with provenance spanning both squads, and completeness propagates *up* the hierarchy: a dropped in-squad witness makes that squad’s partial **incomplete**, which the evidence-aware L3 inherits, whereas a best-effort L3 that ignores squad completeness silently clears the mission. Platform compliance, squad aggregation and mission composition are thus each load-bearing.

7.3 Robustness, Attribution, and Sensitivity

The split is produced by an LLM planner. Given the mission order and the injected intel note, an LLM planner emits the per-platform actions that compose the violation. The architecture *evaluated* here is a single mission planner: one call receives the mission order, the poisoned retrieved note and the action schema, and returns the platform-specific assignments, which are then dispatched to the four platform controllers. That setup gives the attack no information-fragmentation advantage, since the planner observes the complete task allocation. The deployment picture of Sect. 3, with a planner per platform, admits a *distributed* variant of the injection, four independent calls each receiving only its own poisoned retrieval, which we do not evaluate here. The monitors are

unchanged either way, since they consume emitted actions rather than the planner topology. The run we report here was executed on 22 July 2026 over seven planners, four local (`llama3.2`, `qwen2.5:7b`, `llama3.1:8b`, `mistral`) served by Ollama at temperature 0.8 and three hosted (`claude-haiku-4-5-20251001`, `claude-sonnet-5`, `claude-opus-4-8`) with the sampling parameter omitted because those endpoints reject it, 20 trials per condition and four conditions (clean, subtle injection, directive injection, directive injection with a per-planner prompt defence), 560 archived calls in total. Each call is logged with its timestamp, model identifier, parameters, prompt SHA-256 and raw response. Attack-success rate, all four **FacX** fragments appearing across the swarm, is 0% on the clean control for every planner; under directive injection it is 40% (`llama3.2`), 100% (`qwen2.5:7b`, `llama3.1:8b`, `mistral`) and 85% (Haiku), while Sonnet 5 and Opus 4.8 refuse outright (20/20 refusals). A representative emitted action is

```
{"agent": "uav_1", "kind": "collect", "target": "FacX",
  "attr": "coords", "sensor": "mapper"}
```

one of four such actions differing only in platform, fragment and sensor. Susceptibility is therefore model-dependent and not monotone in capability, whereas the compositional monitor detects every split that is *produced* once its witness events are observed. A wider campaign over more planners and delivery variants is beyond the scope of this paper; here we take the attack as given and focus on evidence-aware *detection* under fault.

Provenance, robustness, and causal attribution. Three further experiments sharpen the picture. *Provenance (RQ3)*: across the six detected violation instances for which a ground-truth contributing set is defined, the reported platform set matched it exactly (6/6), so a supervisor always learns *which* platforms jointly breached the policy. *Robustness*: replaying the *benign* mission under the same faults produces 0 false violations (only *unknowns*): evidence loss can withhold a verdict but never fabricate one, so the RV-FABRIC adds no false alarms. *Causal attribution*: ablating individual guarantees (Table 9) shows each fabric signal prevents a distinct class of silent miss, since removing the event-id gap check (G1) reintroduces the miss under *loss* and removing the mission clock (G3) reintroduces it under *jamming*, so the result is attributable to the fabric rather than the scenario. The mission monitor evaluates a 1000-platform swarm in 1.2 ms. *Statistical stability*: over 500 randomized task-split missions (random platform pool, random benign background and timing) the fabric detects every planted violation and, under a randomly dropped witness, silently clears a real violation in 0% of missions versus 100% for the central monitor, the result is a property of the design, not of a particular mission instance.

Threat-model sensitivity. Detection is not brittle to the adversary’s parameters. Sweeping the *split cardinality* of the collective-exfiltration objective (swarm total 1200 B fixed, per-platform local limit 500 B, mission EMCON budget 900 B), a two-way split puts 600 B on each platform and trips the per-platform (L1) limit, while three-, four-, and six-way splits (400, 300, 200 B each) stay locally

Table 9: Guarantee ablation on the intel-package objective: silent false all-clears when a single fabric signal is removed. G1’s event-id gap detects the dropped witness and G3’s mission clock detects the jammed platform, so each removal reintroduces exactly one class of silent miss.

Configuration	drop-witness (loss)	jam-platform (silence)
full fabric	0	0
–G1 (no gap detection)	1	0
–G3 (no mission clock)	0	2

compliant yet trip the cross-platform (L3) EMCON sum. At these thresholds no split cardinality evades both layers at once, so the adversary is squeezed between the per-platform limit and the cross-platform threshold. Beyond loss and jamming, the fault matrix also covers *reordering*, where the fabric’s trusted ingest order prevents the false alarm a naive arrival-order monitor raises, and a *slow consumer*, where independent cursors preserve full detection (zero false all-clears) at bounded latency.

Timing, skew, and benign collaboration. Over 300 jittery missions the mission-clock silence timeout trades jam-detection latency against false-silence flags (≈ 2.9 flags per mission at 2s, ≈ 0.5 at 5s with jamming detected in ≈ 4 s, zero at 10s but ≈ 9 s latency), so 5s is the knee. Under per-platform clock offsets the order-dependent monitors downgrade to *unknown* beyond the order window rather than emitting a false verdict. Across seven legitimate scenarios that superficially resemble the attacks (authorised full-profile collection, the same fragments on an unprotected target, aborted collection, authorised emergency entry, an authorised targeting workflow, sub- k occupancy, within-budget emissions) the monitors raise 0 false alarms, so the composition is correctly conditioned on target, authorisation, completion, and count.

7.4 Systems, Formal-Engine, and Advanced-Fault Validation

Systems validation on the fabric. The L2 transport guarantees were validated *live on the evaluation host* against real Mosquitto + NATS JetStream, with the gateway durable outbox recast as the L2 coordinator. End-to-end latency over the $\text{gen} \rightarrow \text{MQTT} \rightarrow \text{L2} \rightarrow \text{JetStream} \rightarrow \text{L3}$ path ($n=1000$ live events) has a fault-free median of 3.7ms and, under a 10%-drop link with durable retransmit, a 42ms median with no accepted event lost, well within a second-scale mission-decision budget. Once an outbox entry is persisted no accepted verdict is lost and replay yields no duplicate incident under event-id de-duplication; overload surfaces as an explicit health alert before continuity is lost; and independent cursors keep a lagging consumer from stalling the mission verdict. The guarantees hold on the real fabric, not only in the emulation used for the campaign.

Real formal engine. Exported to the real MonPoly first-order past-time engine, a *simplified* coordinated-collection property (≥ 3 distinct platforms collecting on the protected site within 30 s) fires at the third and fourth contributing platform. This is a proxy for P_{pkg} rather than P_{pkg} itself, dropping the fragment-type conjunction and the authorisation cancellation: it establishes that a cross-platform property of this shape is decidable by a genuine formal engine over the fabric, and encoding the full property in the logic remains future work.

Adaptive adversary, latency, and recovery. Because W is set to the mission window in these experiments, so that the RV-FABRIC aggregates over the whole mission, a patient adversary who spreads the four package parts is still caught within the mission; evasion requires spreading collection *across* missions, a documented limit (Sect. 7.6). Under a dropped witness the RV-FABRIC raises a *finite-latency unknown* the instant the event-id gap becomes observable, whereas the central monitor never signals at all. And a withheld *unknown* resolves to *violation* once a dark contributor returns, whereas the central monitor’s blackout all-clear has already been delivered.

No silent all-clears, and graceful degradation. We collapse the entire fault campaign into a single *No-Silent-Clear Rate*,

$$\text{NSCR} = 1 - \frac{\text{silent false all-clears}}{|I_{\text{campaign}}^*|},$$

the fraction of oracle incidents that are *not* silently cleared, i.e. reported as *violation* or honestly downgraded to *unknown*. We prefer this to a “verdict-preservation” count because an honest *unknown* does not preserve the original *violation*; it merely refuses to clear it, which is exactly the safety property we claim. The RV-FABRIC scores 100%, the central monitor 50%, and per-platform guardrails 0%; the violation-versus-*unknown* breakdown is in Table 7. A compound-fault sweep confirms the guarantee is not fragile to fault *count*: as up to eight squads are simultaneously faulted, the RV-FABRIC’s silent-clear count stays 0 and its *unknown* count rises monotonically (graceful degradation), while the central monitor’s silent misses climb to 8. A detection ROC cannot express this, since it treats an honest *unknown* and a silent miss identically.

Stronger baselines, and why NSCR is not enough on its own. NSCR is a safety metric and is trivially satisfied by abstention, so it must be read with the decision figures. Table 10 does that, and adds the baseline a reviewer should ask for: a central monitor that *does* have source sequence numbers and heartbeats, so it observes loss and silence exactly as the RV-FABRIC does, but has no completeness axis and must therefore answer *violation* or *no_violation*. The evidence signals on their own change nothing: resolving missing evidence optimistically reproduces the best-effort monitor exactly, at three silent all-clears out of nine oracle incidents. Resolving pessimistically removes those three, but only by alarming wherever evidence is missing rather than where a witness set was observed: three

Table 10: Baselines and complementary metrics on the intel-package objective over the three-fault campaign (9 oracle incidents), plus the benign mission under the same faults. The *central+seq+hb* rows have the RV-FABRIC’s evidence signals and lack only the completeness axis. *Alarmed* counts oracle incidents reported as violation; for the pessimistic row three of the nine are precautionary alarms raised because evidence was missing, not because a witness set was observed, which is also why that row produces benign false alarms.

Configuration	Alarmed	Silent	NSCR	Decision	Benign
		clears		coverage	false alarms
per-platform guardrails	0/9	9	0%	100%	0
central, best-effort	6/9	3	67%	100%	0
central+seq+hb, optimistic	6/9	3	67%	100%	0
central+seq+hb, pessimistic	9/9	0	100%	100%	4
always-unknown	0/9	0	100%	0%	0
RV-FABRIC (evidence-aware)	6/9	0	100%	67%	0

of its nine alarms are precautionary, and on the benign mission the same rule fires on all four incident-property pairs for which the RV-FABRIC abstains, giving four false alarms. A two-valued monitor must pick one of the two errors. The RV-FABRIC makes neither, at zero silent clears and zero benign false alarms, and pays instead by abstaining on three incidents (67% decision coverage). The trivial always-unknown monitor scores NSCR 100% with 0 detections and abstains on every benign mission, so the coverage and detection columns exclude it where NSCR does not.

7.5 Cost, Summary, and Assurance Workflow

RQ4, cost. The monitors are past-time and stream-based, adding microseconds per event, and the fabric’s delivery and recovery costs are those of the underlying transport (crash recovery linear in the un-acked backlog, 0% loss to 1000 producers). A synthetic canonical-trace scalability experiment, which replicates the event pattern for larger swarms without instantiating ROS 2 or SITL nodes, shows mission-monitor evaluation growing roughly linearly in swarm size and staying under 2 ms to 1000 platforms (approximately 0.01 ms at 4 platforms, 0.14 at 100, 0.6 at 500 and 1.2 at 1000, these being wall-clock measurements that vary slightly between runs). With the 3.7 ms fault-free and 42 ms degraded-link end-to-end medians reported above, the whole path sits orders of magnitude below a second-scale mission-decision budget.

Mission-assurance workflow. The verdicts are an audit trail, not a terminal pass/fail. An *unknown/incomplete* verdict says a decision is withheld for want of evidence rather than granted by default: “we cannot currently certify Sector B” lets a commander hold or re-task, whereas a silent all-clear invites the adversary’s intended action. Since the monitors are decoupled from the flight controllers, the

assurance layer drops onto an existing autonomy stack without touching safety-critical control.

7.6 Threats to Validity

The quantitative results are *controlled invariant-preservation experiments*, isolating whether each design element preserves the verdict under fault, not a population-level measurement of operational detection accuracy; the clean 0/100% figures should be read in that light. Of the realism layer, the ROS 2/DDS path and the ArduPilot SITL flight dynamics are both *executed* (Table 5); Gazebo integration and a larger at-scale physical sweep are planned and not claimed here. The semantic-extraction path is secondary, so text-only violations depend on classifier quality. Platform integrity beyond going silent is out of scope, and Proposition 1 covers monotone-witness properties, so other property classes remain open.

8 Related Work

Agent safety and multi-step attacks. Single-agent guardrails check one agent’s actions against rules, but tool-attack chains and prompt-injection benchmarks show that many multi-step attacks defeat per-event checks [21,11]. The present work lifts our single-agent composition to the *swarm*, where the violation is distributed across platforms and no single-agent view suffices.

Distributed and spatial RV. Reaching a verdict across faulty, asynchronous monitors is studied at the algorithm level [16,9], and spatio-temporal logics monitor properties over networks of components [3]. That line makes the monitoring *logic* fault- and reordering-tolerant while abstracting the transport. We are complementary and beneath it: the RV-FABRIC makes the delivery layer’s loss, ordering and liveness observable, so the stream a swarm monitor consumes is the stream those algorithms assume. Our hierarchical monitoring line established these layers for edge-IoT security and cyber-physical resilience [19,18,20,25]; this paper carries the hierarchy to a cooperative LLM-assisted swarm.

Runtime assurance for robotics, and M&S. Runtime-assurance architectures switch to a verified safety controller when a monitor flags danger, from Simplex [24] to robotics frameworks such as SOTER [12]; ROS-level monitors attach RV to a robot’s message bus [15]. These secure a *single* platform’s safety envelope, whereas our concern is a mission-level, cross-platform obligation no single monitor can express, checked over an evidence-aware transport. Modelling and simulation is the established setting for evaluating such assurance under repeatable, injected conditions [22]; digital-twin and FMI co-simulation approaches assure autonomous systems against a high-fidelity model [28,27,1], within a continuous-engineering methodology for trustworthy learning-enabled

systems [8], and design-time analysis can check learning-enabled components directly [13]. Our contribution is complementary to both: a simulation-evaluated, compositional, evidence-aware monitor for the swarm’s *mission-level* obligations, whose verdicts are auditable assurance evidence rather than a pass/fail flag.

9 Conclusion

We presented a three-layer, evidence-aware compositional runtime-verification framework for LLM-assisted robot swarms on ISR missions in contested environments. A prompt-injection attack compliant at every platform is invisible to per-platform monitors yet detected with provenance by the compositional monitor, and under loss and jamming the fabric emits zero silent false all-clears where a best-effort central monitor emits several. The framework lifts hierarchical agent-safety monitoring from the single agent to the swarm and turns transport guarantees into auditable, platform-attributed assurance evidence. Future work adds Gazebo integration, source-side sequence numbering to carry the guarantee across the contested link, a larger at-scale sweep over the executed ROS2/ArduPilot layer, and soundness beyond the monotone-witness class.

References

1. Abdelsalam, M., Bensalem, S., Delacourt, A., He, W., Katsaros, P., Kekatos, N., Ruiz Nolasco, R., Peled, D., Ponchant, M., Ryad, I., Temperekidis, A., Wu, C.: Digital twin for the formal analysis of a depth of anesthesia controller. *SIMULATION* **101**(3), 341–360 (2025)
2. Ahn, M., Brohan, A., Brown, N., et al.: Do as i can, not as i say: Grounding language in robotic affordances. In: *Conf. on Robot Learning (CoRL)* (2022)
3. Bartocci, E., Bortolussi, L., Loret, M., Nenzi, L.: Monitoring mobile and spatially distributed cyber-physical systems (STREL). In: *MEMOCODE*. pp. 146–155 (2017)
4. Bartocci, E., Falcone, Y., Francalanza, A., Reger, G.: Introduction to runtime verification. *Lectures on Runtime Verification, LNCS 10457* (2018)
5. Basin, D., Klaedtke, F., Müller, S., Zălinescu, E.: Monitoring metric first-order temporal properties. *Journal of the ACM* **62**(2) (2015)
6. Basin, D., Klaedtke, F., Zălinescu, E.: The MonPoly monitoring tool. In: *RV-CuBES* (2017)
7. Bauer, A., Leucker, M., Schallhart, C.: Runtime verification for LTL and TLTL. *ACM Transactions on Software Engineering and Methodology (TOSEM)* **20**(4) (2011)
8. Bensalem, S., Katsaros, P., Kekatos, N., et al.: Continuous engineering for trustworthy learning-enabled autonomous systems. In: *Bridging the Gap Between AI and Reality (AISO LA 2023)*. LNCS, vol. 14380. Springer (2023)
9. Bonakdarpour, B., Fraigniaud, P., Rajsbaum, S., Rosenblueth, D.A., Travers, C.: Decentralized asynchronous crash-resilient runtime verification. In: *CONCUR* (2016)
10. Chung, S.J., Paranjape, A.A., Dames, P., Shen, S., Kumar, V.: A survey on aerial swarm robotics. *IEEE Transactions on Robotics* **34**(4), 837–855 (2018)

11. Debenedetti, E., et al.: AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. In: *NeurIPS Datasets and Benchmarks* (2024)
12. Desai, A., Ghosh, S., Seshia, S.A., Shankar, N., Tiwari, A.: SOTER: A runtime assurance framework for programming safe robotics systems. In: *49th IEEE/IFIP Int. Conf. on Dependable Systems and Networks (DSN)*. pp. 138–150 (2019)
13. Eleftheriadis, C., Kekatos, N., Katsaros, P., Tripakis, S.: On neural network equivalence checking using SMT solvers. In: *Formal Modeling and Analysis of Timed Systems (FORMATS)*. pp. 237–257. LNCS (2022)
14. Faymonville, P., Finkbeiner, B., Schledjewski, M., Schwenger, M., Stenger, M., Tentrup, L., Torfah, H.: StreamLAB: Stream-based monitoring of cyber-physical systems. In: *Computer Aided Verification (CAV)* (2019)
15. Ferrando, A., Cardoso, R.C., Fisher, M., Ancona, D., Franceschini, L., Mascardi, V.: ROSMonitoring: A runtime verification framework for ROS. In: *Towards Autonomous Robotic Systems (TAROS)*. LNCS, vol. 12228, pp. 387–399. Springer (2020)
16. Fraigniaud, P., Rajsbaum, S., Travers, C.: On the number of opinions needed for fault-tolerant run-time monitoring in distributed systems. In: *Runtime Verification (RV)* (2014)
17. Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., Fritz, M.: Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In: *ACM Workshop on Artificial Intelligence and Security (AISec)* (2023)
18. Kekatos, N., Chintri, M., Katsaros, P., Nianios, T., Seitoglou, I., Basagiannis, S.: Hierarchical security monitoring for edge-iot: A formal methods approach. In: *IEEE International Conference on Cyber Security and Resilience (CSR)* (2026)
19. Kekatos, N., Chintri, M., Katsaros, P., Lekidis, A., Nianios, T., Seitoglou, I., Temperekidis, A., Basagiannis, S.: A hierarchical runtime-verification approach for security monitoring of edge-iot. In: *Int. Workshop on Advances in Practical Security (ARES)* (2026)
20. Kekatos, N., Koutidis, G., Antonakopoulos, K., Basagiannis, S., Katsikas, S., Kavalieratos, G., Lekidis, A., Nianios, T., Papageorgiou, E.: RESILAGENT: A three-pillar architecture for cyber-physical resilience in electrical power and energy systems. In: *ARES 2026 Workshops (EPES-SPR)* (2026)
21. Li, J.J., He, J., Shang, C., Kulshreshtha, D., Xian, X., Zhang, Y., Su, H., Swamy, S., Qi, Y.: STAC: When innocent tools form dangerous chains to jailbreak LLM agents (2025), arXiv:2509.25624
22. Mazal, J., Fagiolini, A., et al. (eds.): *Modelling and Simulation for Autonomous Systems (MESAS 2024)*, Revised Selected Papers, LNCS, vol. 15761. Springer (2025)
23. OASIS: MQTT version 5.0. OASIS Standard (2019)
24. Sha, L.: Using simplicity to control complexity. *IEEE Software* **18**(4), 20–28 (2001)
25. Sinha, P., Kekatos, N., Basagiannis, S., Bruto da Costa, A.A., Lekidis, A., Mitra, P., Nianios, T., Papageorgiou, E.: Explainable rule mining of IPv6 extension-header presence patterns from paired-vantage captures. In: *ARES 2026 Workshops (GenXSec)* (2026)
26. Synadia / CNCF: NATS and JetStream. <https://nats.io> (2024)
27. Temperekidis, A., Kekatos, N., Katsaros, P.: Runtime verification for FMI-based co-simulation. *Runtime Verification (RV)*, LNCS 13498 (2022)
28. Temperekidis, A., Kekatos, N., Katsaros, P., et al.: Towards a digital twin architecture with formal analysis capabilities for learning-enabled autonomous systems. In: *MESAS*. LNCS, vol. 13866 (2022)